

Machine Learning

Capstone Project – 1

Introduction

In this machine learning capstone project, I select the Singapore Housing and Development Board's (HDB) resale price data set. This dataset can be got from <https://data.gov.sg/collections/189/view>

Best Practices

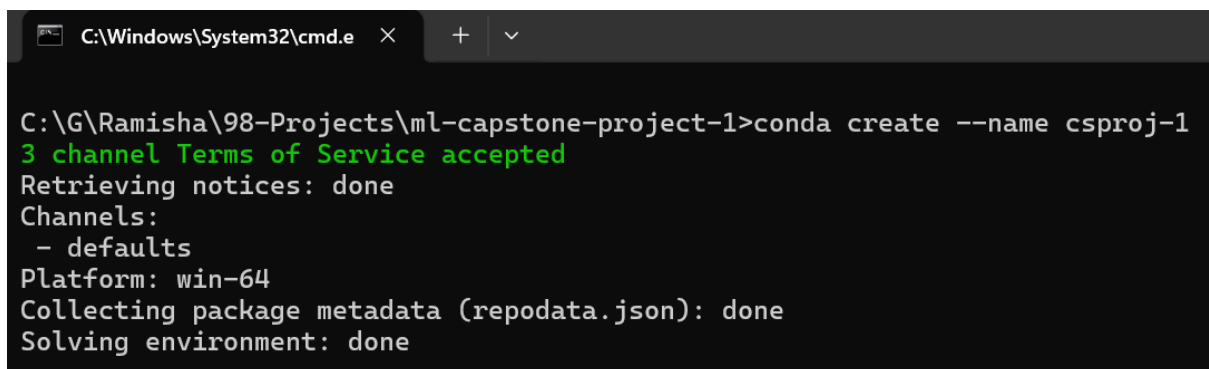
Its best practice is to do the project in each virtual machine to avoid the dependency issues. Follow the steps to create the VM.

Create virtual environment

```
conda create --name csproj-1
activate csproj-1
```

Install necessary dependencies

```
conda install python=3.12
conda install numpy==2.4.6
conda install pandas
conda install matplotlib
conda install scikit-learn==1.9.0
conda install jupyter notebook
```



```
C:\Windows\System32\cmd.e  X  +  v

C:\G\Ramisha\98-Projects\ml-capstone-project-1>conda create --name csproj-1
3 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

This project dissertation has been described in following sections.

1. Data collection

Please refer: [01-Data-Collection.ipynb](#)

Dataset has **232,188** rows and **15** columns; the table below briefs the columns.

#	X/Y VARIABLES	COLUMNS	DATA TYPE	DESCRIPTION
1	Feature Variables	month	Categorical	Sold month (eg. 2017-01)
2		town	Categorical	Town (eg. Queenstown, Ang Mo Kio)
3		flat_type	Categorical	Flat type (eg. 2-room, 3-room)
4		block	Numerical	Block number of the unit (eg. 82)
5		street_name	Categorical	Street name
6		storey_range	Categorical	Low floor, Mid floor, High floor
7		floor_area_sqm	Numerical	Floor size in square meter
8		floor_area_sqft	Numerical	Floor size in square feet
9		flat_model	Categorical	Type of flat model
10		lease_commence_date	Numerical	Apartment build year
11		house_age	Numerical	As of now house's age
12		remaining_lease	Numerical	Remaining lease year (total 99 years)
13		price_per_sqm	Numerical	Price per square meter
14		price_per_sqft	Numerical	Price per square feet
15	Target Variable	resale_price		Sold price (This will be our target variable)

If you see the dataset, it's features dealing with numerical and categorical data the domain selection will be **Machine Learning**, also the features and target variable clearly available, we choose the learning type as **Supervised Learning**, finally the target variable is numerical value, which is resale_price, we go with **regression** type.

Stage 01 - Domain Selection = Machine Learning

Stage 02 - Learning Type = Supervised Learning

State 03 - Regression/Classification = Regression

2. Data Science - Univariate & Bivariate analysis

While the end goal of the AI is prediction of the future data, the goal of data science is analyzing past data. The following questions will help us to understand the dataset better.

To simplify I created a python file named **hopelib.py** and created a HDB class with functions. Please refer to hopelib.py in the repo.

I. Get all the towns in the dataset.

```
from hopelib import HDB
hdb = HDB(dataset)
```

Initialize HDB class

```
# Get all towns in the dataset
hdb.get_all_towns()
```

['ANG MO KIO', 'BEDOK', 'BISHAN', 'BUKIT BATOK', 'BUKIT MERAH', 'BUKIT PANJANG', 'BUKIT TIMAH', 'CENTRAL AREA', 'CHOA CHU KANG', 'CLEMENTI', 'GEYLANG', 'HOUGANG', 'JURONG EAST', 'JURONG WEST', 'KALLANG/WHAMPOA', 'MARINE PARADE', 'PASIR RIS', 'PUNGGOL', 'QUEENSTOWN', 'SEMBAWANG', 'SENGKANG', 'SERANGOON', 'TAMPINES', 'TOA PAYOH', 'WOODLANDS', 'YISHUN']

II. Get all flat types

```
# Get all flat types
hdb.get_flat_types()
```

```
['2 ROOM',
 '3 ROOM',
 '4 ROOM',
 '5 ROOM',
 'EXECUTIVE',
 '1 ROOM',
 'MULTI-GENERATION']
```

III. Get flat models

```
# Get flat models
hdb.get_flat_models()
```

['Improved',
'New Generation', 'DBSS', 'Standard', 'Apartment', 'Simplified', 'Model A',
'Premium Apartment', 'Adjoined flat', 'Model A-Maisonette', 'Maisonette',
'Type S1', 'Type S2', 'Model A2', 'Terrace', 'Improved-Maisonette', 'Premium
Maisonette', 'Multi Generation', 'Premium Apartment Loft', '2-room', '3Gen']

IV. Retrieve dataset for town (eg. Town = BUKIT BATOK). It will return the data only belongs to the given town.

```
# Grab the dataset for particular town only
bukit_batok_dataset = hdb.get_dataset_by_town("BUKIT BATOK")
```

```
bukit_batok_dataset.head()
```

	month	town	flat_type	block	street_name	storey_range	floor_area_sqm	floor_area_sqft
148	2017-01	BUKIT BATOK	3 ROOM	342	BT BATOK ST 34	01 TO 03	64.0	688.8896
149	2017-01	BUKIT BATOK	3 ROOM	530	BT BATOK ST 51	04 TO 06	64.0	688.8896

V. Get average price in the town

```
price = hdb.get_average_price_by_town('BUKIT BATOK')
print("Average price in BUKIT BATOK: S$", round(price, 2))
```

Average price in BUKIT BATOK: S\$ 507369.82

```
price = hdb.get_average_price_by_town('QUEENSTOWN')
print("Average price in QUEENSTOWN: S$", round(price, 2))
```

Average price in QUEENSTOWN: S\$ 644124.51

VI. Get the highest and lowest prices in the town.

```
# Highest price in the town
highest_price = hdb.get_highest_price_by_town('BUKIT BATOK')
print("Highest price in BUKIT BATOK: S$", round(highest_price, 2))
```

Highest price in BUKIT BATOK: S\$ 1180000.0

```
# Lowest price in the town
lowest_price = hdb.get_lowest_price_by_town('BUKIT BATOK')
print("Lowest price in BUKIT BATOK: S$", round(lowest_price, 2))
```

Lowest price in BUKIT BATOK: S\$ 208000.0

3. Data pre-processing

Since model does not understand the categorical data, pre-processing is the crucial step before passing the data into model.

Remove the less importance features from the dataset.

```
# Drop less important columns
dataset.drop(columns=["month", "block", "street_name", "remaining_lease"], inplace=True)
```

Then convert the categorical data to numerical values using panda's `get_dummies` method.

```
# Since model cannot understand the categorical data, needs to encode before processing.
dataset = pd.get_dummies(dataset, dtype=int, drop_first=True)
```

```
dataset
```

Then divide the features and target variables.

```
X = dataset.drop("resale_price", axis=1)
y = dataset["resale_price"]
```

After split the training and test dataset.

```
# Split data into training and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Finally in the pre-processing, standardize the numerical data using StandardScaler.

```
scaler_x = StandardScaler()
X_train = scaler_x.fit_transform(X_train)
X_test = scaler_x.transform(X_test)

# Use the same scaler in deployment phase, to avoid redundant code
filename = "hdb_scaler_x.sav"
pickle.dump(scaler_x, open(filename, "wb"))
```

```
scaler_y = StandardScaler()
y_train = scaler_y.fit_transform(
    np.array(y_train).reshape(-1, 1)
).ravel()

y_test = scaler_y.transform(
    np.array(y_test).reshape(-1, 1)
).ravel()

filename = "hdb_scaler_y.sav"
pickle.dump(scaler_y, open(filename, "wb"))
```

Standardizing the target variable is depends on the performance. If the model gives a good r2 score, then prefer to standardize target variable as well. This case I did for X and Y variables.

When the deployment phases, the result also must inverse transform before passing to user, to do this please save the x and y's scalers in file using Python's pickle library.

4. Model creation

Please refer to: [2-Pre-Processing-Model-Creation.ipynb](#)

Initially I was planning to create 3 models

- DecisionTreeRegressor
- RandomForestRegressor
- SVR

But SVR model took long time processing I skip and create only DecisionTreeRegressor and RandomForestRegressor.

Also creating models with random value is not good and its only for flow testing purposes. So I went through GridSearchCV and model itself pick the best parameters (for both models).

```
# Model 1: DecisionTreeRegressor
```

```
grid = {
    "criterion": ['squared_error', 'friedman_mse'],
    "max_features": ["sqrt"],
    "splitter": ["best", "random"]
}

model_dt = GridSearchCV(DecisionTreeRegressor(), grid, refit = True, verbose = 3, n_jobs = 1)
model_dt.fit(X_train, y_train)
```

```
# Model 2: RandomForestRegressor
```

```
grid = {
    "criterion": ['squared_error', 'friedman_mse'],
    "n_estimators": [10, 25, 50],
    "max_features": ["sqrt", "log2"]
}

model_rf = GridSearchCV(RandomForestRegressor(), grid, refit = True, verbose = 3, n_jobs = 1)
model_rf.fit(X_train, y_train)
```

After learning phase completed, save the models in the file.

```
# Save the model for deployment purpose
pickle.dump(model_dt, open("hdb_model_dt.sav", "wb"))
pickle.dump(model_rf, open("hdb_model_rf.sav", "wb"))
# pickle.dump(model_svr, open("hdb_model_svr.sav", "wb"))
```

5. Model evaluation and finalizing

After models built, needs to evaluate the performance. For regression models we can use `r2_score`.

Both DecisionTree and RandomForest models scored nearly 0.99% and RandomForest scored bit more.

```
# R2 Score -> DecisionTreeRegressor
y_pred = model_dt.predict(X_test)
score_dt = r2_score(y_test, y_pred)
print("R2 Score for DecisionTreeRegressor:", score_dt)

R2 Score for DecisionTreeRegressor: 0.9918730779657848
```

```
# R2 Score -> RandomForestRegressor
y_pred = model_rf.predict(X_test)
score_rf = r2_score(y_test, y_pred)
print("R2 Score for RandomForestRegressor:", score_rf)

R2 Score for RandomForestRegressor: 0.9988686184110437
```

6. Deployment with Notebook

It's deployment time. Since we scaled the X & Y variables we must do the same process to user input data as well. In the pre-processing phase we saved both of these scalers in the file and now can utilize using Python's pickle lib.

```
scaler_x = pickle.load(open("hdb_scaler_x.sav", "rb"))
scaler_y = pickle.load(open("hdb_scaler_y.sav", "rb"))
```

scaler_x

▼ StandardScaler ⓘ ?

► Parameters

scaler_y

▼ StandardScaler ⓘ ?

► Parameters

```
scaled_data_x = scaler_x.transform(input_data_df)
```

Also, the load the model in the same way.

```
# Load the finalized model
model = pickle.load(open("hdb_model_rf.sav", "rb"))
```

```
price = model.predict(scaled_data_x)
price
```

```
array([-0.51107495])
```

```
price.shape
```

```
(1,)
```

```
inversed_price = scaler_y.inverse_transform(price.reshape(-1, 1))
inversed_price
```

```
array([[432833.28]])
```

Its important that need to reverse the output value again using the same scale's output.

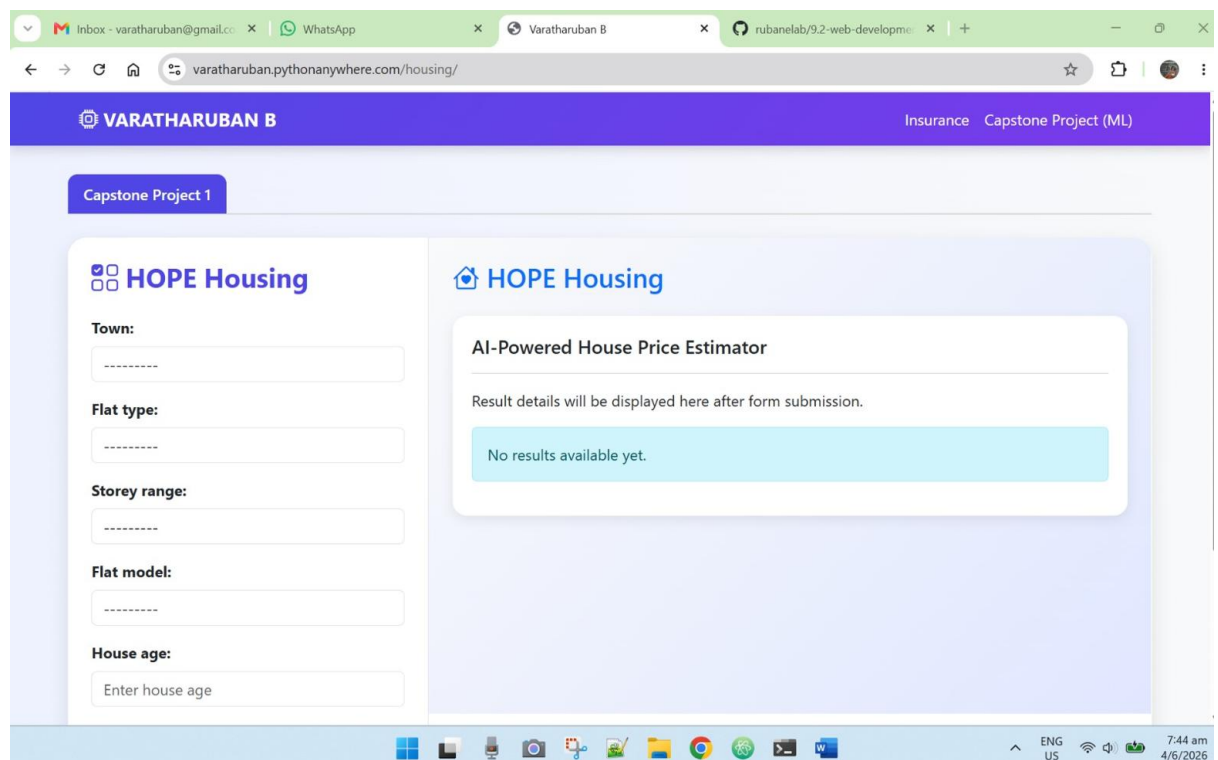
7. Deployment with Django

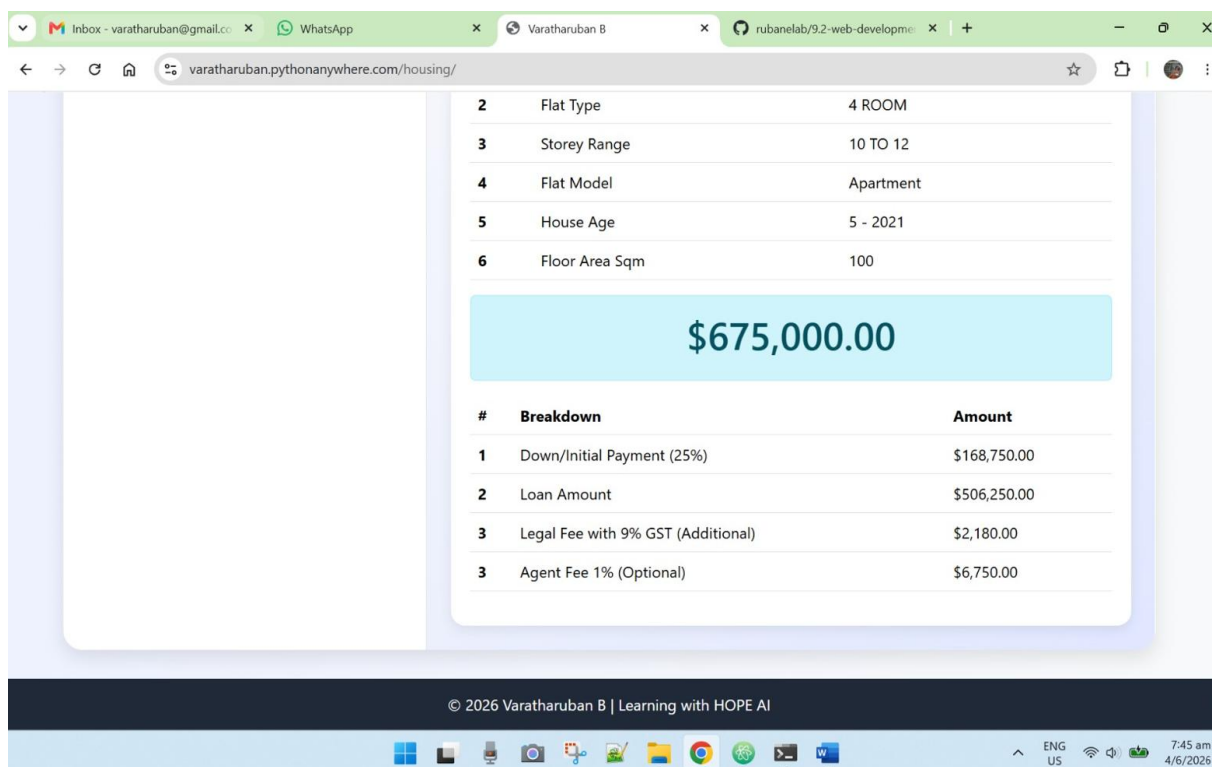
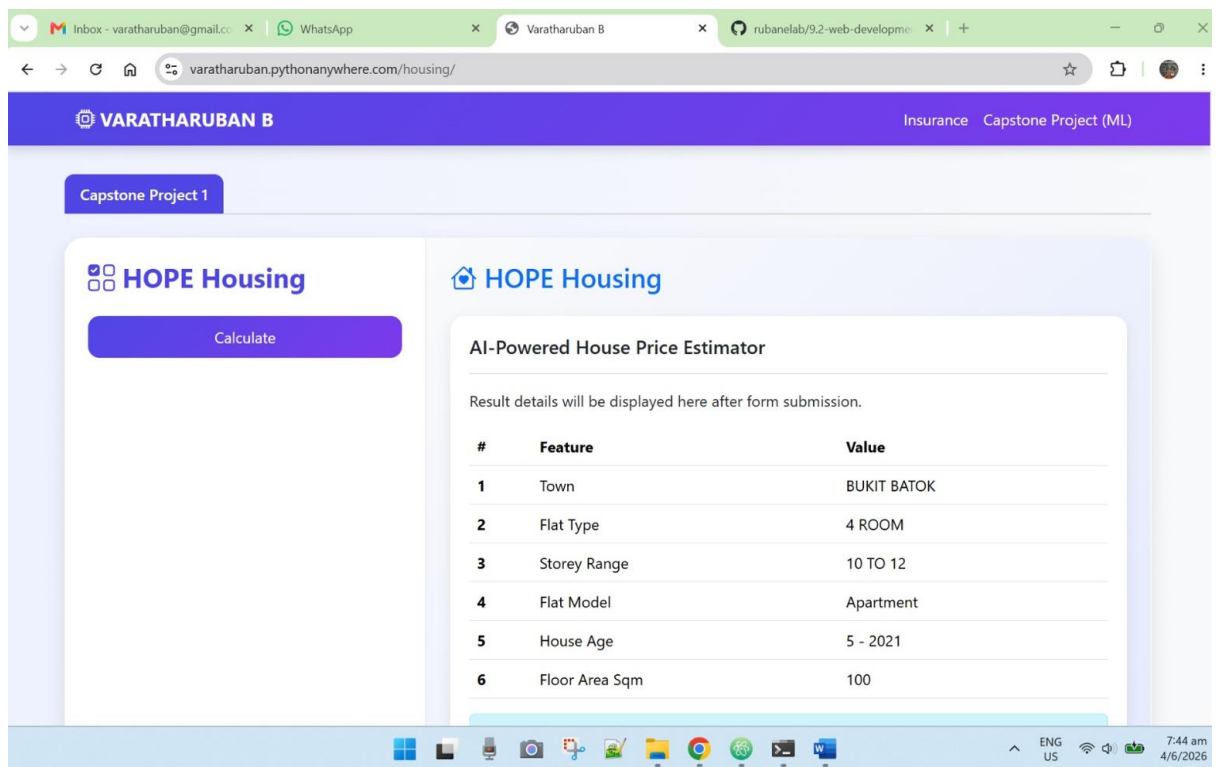
For the end-user experience, this process developed in a web application and hosted into PythonAnywhere platform.

Please visit this URL for a live experience.

<https://varatharuban.pythonanywhere.com/housing/>

Some screenshots.





08:15

uban.pythonanywhere.com

6

Flat model:

House age:

Enter house age

Floor area sqm:

Enter floor size in sq.m

Calculate

 HOPE Housing

AI-Powered House Price Estimator

Result details will be displayed here after form submission.

No results available yet.



Varatharuban B

Machine Learning

Capstone Project - 1

AI-Powered House Price Estimator

Result details will be displayed here after form submission.

#	Feature	Value
1	Town	QUEENSTOWN
2	Flat Type	5 ROOM
3	Storey Range	19 TO 21
4	Flat Model	DBSS
5	House Age	5 - 2021
6	Floor Area Sqm	1200

\$700,000.00

#	Breakdown	Amount
1	Down/Initial Payment (25%)	\$175,000.00
2	Loan Amount	\$525,000.00
3	Legal Fee with 9% GST (Additional)	\$2,180.00

